MBLP131 · Hafta 13 · Ders Notları

İşletim Sistemleri Temelleri

Süreçler ve İş Denetimi

Yazdığınız her komut bir sürece dönüşür. Bu hafta onu izlemeyi, duraklatmayı, taşımayı ve durdurmayı öğreniyorsunuz.

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. Kabuğun her komutu `fork()`, `exec()` ve `wait()` ile nasıl bir sürece dönüştürdüğünü açıklamak; sürecin PID'sini ve PPID'sini bulmak.
2. Süreçleri `ps` (`-f`, `-ef`, `aux`, `-o`), `pstree`, `top`/`htop` ve `/proc/<pid>/status` ile listelemek ve incelemek; yük ortalaması dahil her sütunu okumak.
3. 3. haftanın beş süreç durumunu STAT harfleri R, S, D, T ve Z ile eşleştirmek.
4. İşleri ön planda ve arka planda çalıştırmak; &, Ctrl+Z, Ctrl+C, `jobs`, `fg` ve `bg` ile aralarında taşımak.
5. `kill`, `pkill` ve `killall` ile sinyal göndermek; SIGTERM, SIGKILL, SIGINT, SIGHUP, SIGSTOP ve SIGCONT sinyallerini ve $128 + n$ çıkış durumunu açıklamak.
6. `nohup`, `$$` ve `$!` kullanmak; öncelikleri `nice` ve `renice` ile değiştirmek ve işlemci paylarını öngörmek (5. hafta).
7. Bilerek bir zombi süreç oluşturmak, onu tanımak ve neden yalnızca ebeveynin onu kaldırabildiğini açıklamak.

İçindekiler

1 12. Haftadan 13. Haftaya 1
2 Süreçlere Yeniden Bakış 2
3 Süreçlere Bakmak 2
4 İş Denetimi 8
5 Sinyaller 10
6 Öncelikler: `nice` ve `renice` 15

7 Zombiler ve Yetimler, Görünür Hâlde 16

8 Komut Başvuru Tablosu 18

9 Sık Yapılan Hatalar ve Anlamları 18

10 Özet 19

11 Sözlük 19

12 Kendiniz Deneyin 20

13 Tekrar Soruları 20

14 İleriye Bakış: 14. Hafta 21

15 Laboratuvar Föyü 22

16 Laboratuvar Föyü Çözümleri 23

1 12. Haftadan 13. Haftaya

Geçen hafta programları yönlendirmeler ve borularla birbirine bağladınız. `grep ERROR logs/system.log | wc -l` gibi her satırın arkasında kabuk sessizce *süreçler* oluşturdu: aynı anda çalışan ve bir boruyla birleştirilmiş iki süreç. Bu hafta bu süreçlere doğrudan bakıyoruz. Gerekenlerin neredeyse tamamı kâğıt üzerinde 3. haftada (süreç, PID, süreç durumları, PCB, `fork()`/`exec()`/`wait()`, zombiler) ve 5. haftada (öncelikli zamanlama, nice değerleri) tanıtılmıştı. Şimdi bu fikirlerin her birini gerçek bir Linux terminalinde görecek ve süreçleri kendiniz denetlemeyi öğreneceksiniz.

Bu notlardaki çıktılar hakkında. Burada gösterilen her oturum gerçek çıktıdır; laboratuvar makinemizde, sıradan kullanıcı `student` olarak ve yeni oluşturulmuş bir `~/lab` klasöründe yakalandı. Komutlar `code/demo_*.sh` dosyalarında, yakalanan çıktılar ise `code/out_*.txt` dosyalarındadır. İş denetimi (job control; Ctrl+Z, `fg`, `bg`) yalnızca bir terminale sahip *etkileşimli* bir kabukta çalışır; bu yüzden gösterimler küçük bir yardımcı program (`code/record_session.py`) tarafından gerçek bir terminale yazıldı. `demo` dosyalarını "bu satırları tek tek yazın" diye okuyun. Bu dosyalarda `#@` ile başlayan satırlar "Ctrl+Z'ye basın" gibi yönergelerdir.

Sizin makinenizde PID'ler, saatler, tarihler, terminal adları (`pts/0`) ve kullanıcı numaraları farklı olacak; PID'ler her çalıştırmada değişir. Makinedeki bütün süreçleri içeren listeler (`ps -ef`, `ps aux`) *kırpıldı*: ilk satırları ve yalnızca `student` kullanıcısının süreçlerini gösteriyor, çok uzun satırları "…" ile kesiyoruz. `ls` ve `sort` çıktısının sırası *yerel ayara* (locale) bağlı olabilir; biz `LC_ALL=C.UTF-8` kullandık. Türkçe yerel ayarlı bir sistemde (`LANG=tr_TR.UTF-8`) birçok ileti Türkçe görünebilir ve sıralama farklı olabilir; bu notlardaki İngilizce iletiler, programların C yerel ayarında yazdığı biçimdedir.

Bu hafta her yerde iki zararsız deneme programı kullanıyoruz: `sleep N`, N saniye bekler ve hiçbir şey yapmaz (ömrünü *bekliyor* durumunda geçirir); `yes` ise sonsuza dek y harfini yazar. `yes > /dev/null` ile

çıktısı atılır ve yalnızca bir işlemci çekirdeğini meşgul eder (her zaman *hazır* ya da *çalışıyor* durumundadır).

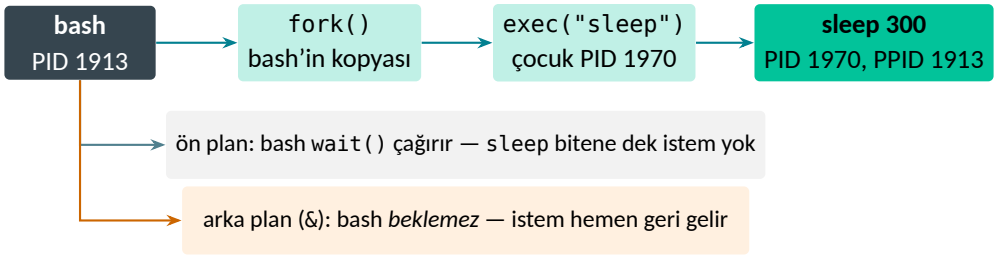
2 Süreçlere Yeniden Bakış

3. haftadan hatırlayın: **program**, diskte duran edilgen bir dosyadır (örneğin `/usr/bin/sleep`); **süreç** (process) ise yürütülmekte olan bir programdır. Her sürecin benzersiz bir **süreç kimliği** (process identifier, PID) vardır, ebeveyninin PID'sini (PPID) bilir ve çekirdekte **süreç denetim bloğu** (process control block, PCB) ile tanımlanır. Süreçler bir ağaç oluşturur, çünkü (ilki dışında) her biri bir başkası tarafından oluşturulur.

Bir komut yazdığınızda kabuk (`bash`), 3. haftadaki C programı `fork_exec_wait.c`'nin yaptığının aynısını yapar:

1. `fork()` kabuğun bir kopyasını, yani çocuğu oluşturur;
2. çocuk `exec()` çağırır; bu çağrı `bash` kopyasını istediğiniz programla değiştirir, PID aynı kalır;
3. ebeveyn kabuk `wait()` çağırır ve sonraki komut istemini (prompt) ancak çocuk bittiğinde yazdırır.

Komut & ile bitiyorsa 3. adım atlanır: kabuk beklemez, istemi hemen yazdırır ve komut *arka planda* çalışır. Ön plan ile arka plan komutu arasındaki farkın tamamı budur.



Şekil 1: Kabuk `sleep 300 &` komutunu nasıl çalıştırır? PID'ler Kısım 3'teki gerçek oturumdan alınmıştır.

3 Süreçlere Bakmak

3.1 `ps`: süreçlerinizin anlık görüntüsü

`ps` (*process status*) süreçlerin tek seferlik bir anlık görüntüsünü yazdırır. Seçeneksiz olarak yalnızca geçerli terminalinize bağlı süreçleri gösterir. `$$`, kabuğun kendi PID'sini tutan özel bir kabuk değişkenidir.

```
student@okan-lab:~/lab$ echo $$
1913
student@okan-lab:~/lab$ ps
  PID TTY          TIME CMD
 1913 pts/0    00:00:00 bash
 1969 pts/0    00:00:00 ps
student@okan-lab:~/lab$ sleep 300 &
[1] 1970
student@okan-lab:~/lab$ yes > /dev/null &
[2] 1971
student@okan-lab:~/lab$ ps -f
UID        PID  PPID  C STIME TTY          TIME CMD
student   1913  1912  0 16:58 pts/0    00:00:00 -bash
student   1970  1913  0 16:58 pts/0    00:00:00 sleep 300
student   1971  1913 96 16:58 pts/0    00:00:00 yes
student   1973  1913  0 16:58 pts/0    00:00:00 ps -f
```

Neye dikkat etmeli?

- Kabuğumuzun PID'si 1913'tür; listede `bash` olarak görünür, `ps -f` ise tam adını, `-bash`'i gösterir. Baştaki tire bir *giriş kabuğunu* (login shell) belirtir.
- `ps` kendisini de listeler: anlık görüntüyü aldığı sırada o da çalışan bir süreçtir. (İki arka plan komutundan sonraki ikinci yalın `ps`, burada atlandı, `bash`, `sleep`, `yes` ve `ps` süreçlerini listeler.)
- İki arka plan komutu `[1] 1970` ve `[2] 1971` yazdırır: köşeli parantez içinde bir *iş numarası* (job number) ve PID.
- `ps -f` (*full*, tam biçim) sahibi (UID), PPID'yi, işlemci kullanımı C'yi, başlama zamanını ve komut satırının tamamını ekler. `sleep` ve `yes` süreçlerinin PPID'si 1913'tür: ebeveynleri kabuktur. Kabuğun PPID'si olan 1912, onu başlatan terminal programına aittir.
- `yes` sürecinin `C` sütunu 96'dır: bir işlemci çekirdeğinin neredeyse tamamını kullanır. PID'ler atlar (1913'ten 1969'a), çünkü bu arada başka kullanıcılar süreç oluşturdu.

3.2 `-o` ile sütun seçmek ve `pstree`

`-o` seçeneği tam olarak ihtiyacınız olan sütunları seçmenizi sağlar. En sık kullandığımız biçim budur:

```
student@okan-lab:~/lab$ ps -o pid,ppid,stat,ni,%cpu,%mem,cmd
  PID  PPID STAT  NI %CPU %MEM CMD
 1913  1912 Ss     0  0.5  0.0 -bash
 1970  1913 S      0  0.0  0.0 sleep 300
 1971  1913 R      0 97.4  0.0 yes
 1974  1913 R+     0  0.0  0.0 ps -o pid,ppid,stat,ni,%cpu,%mem,cmd
student@okan-lab:~/lab$ pstree -p $$
bash(1913)─┬─pstree(1975)
           ├─sleep(1970)
           └─yes(1971)
```

`STAT` süreç durumudur (Kısım 3.5), `NI` ise nice değeri (Kısım 6). `yes` `R` durumundadır ve işlemcinin %97,4'ünü kullanır; `sleep` `S` durumundadır ve %0 kullanır. `pstree -p $$`, 3. haftanın süreç ağacını kabuğumuzdan başlayarak gerçek PID'lerle çizer: kabuğun üç çocuğu vardır. Oturumun sonunda iki işi de durdurduk:

```
student@okan-lab:~/lab$ kill %1 %2
student@okan-lab:~/lab$ jobs
[1]-  Terminated              sleep 300
[2]+  Terminated              yes > /dev/null
```

`%1` "1 numaralı iş" demektir (Kısım 4); kabuk her iş için bir kez `Terminated` bildirir.

3.3 Bütün süreçler: `ps -ef`, `ps aux`, `ps -u`

Tarihsel nedenlerle iki seçenek stili vardır. `ps -ef` (System V stili, tireli) ve `ps aux` (BSD stili, tire*siz*) ikisi de *bütün* kullanıcıların *bütün* süreçlerini listeler; `aux` bellek sütunlarını ekler. Laboratuvar makinemizde tam listeler, çoğu `root`'a ait yaklaşık 70 satırdır; bu yüzden yalnızca `ps -ef`'in başını gösteriyor, ardından `student` kullanıcısını `grep` ile seçiyoruz (`ps -f -u student` aynı seçimi `ps -f` biçiminde verir):

```
student@okan-lab:~/lab$ ps -ef | head -4
UID          PID  PPID  C STIME TTY          TIME CMD
root           1     0  0 16:17 ?        00:00:02 /process_api --firecracker-init --addr
```

```
    0.0.0.0:2024 --max-ws-buffer-size 32768 --block-local-connections --listen-vsock-port 2024
     --log-vsock-port 5002
root         2     0  0 16:17 ?        00:00:00 [kthreadd]
root         3     2  0 16:17 ?        00:00:00 [pool_workqueue_release]
student@okan-lab:~/lab$ ps aux | head -1
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
student@okan-lab:~/lab$ ps aux | grep '^student'
student   1913  0.6  0.0   8436  7532 pts/0    Ss   16:58   0:00 -bash
student   1970  0.0  0.0   3132  1996 pts/0    S    16:58   0:00 sleep 300
student   1971 99.0  0.0   3132  1884 pts/0    R    16:58   0:02 yes
student   1981  0.0  0.0   8340  4328 pts/0    R+   16:58   0:00 ps aux
student   1982  0.0  0.0   4100  2200 pts/0    S+   16:58   0:00 grep ^student
```

- **PID 1**, açılışta çekirdeğin başlattığı ilk kullanıcı alanı sürecidir; yetimleri evlat edinir (3. hafta). Laboratuvar makinemizde bu, `process_api` adlı küçük bir konteyner başlatma programıdır (uzun komut satırı yukarıda alt satıra kayıyor); normal bir Ubuntu kurulumunda PID 1 `systemd`'dir.
- **PID 2**, `kthreadd`, bütün çekirdek iş parçacıklarının ebeveynidir; bunların adları köşeli parantez içinde gösterilir.
- `ps aux | grep '^student'` öğrencinin satırlarını bulur — ve `ps` anlık görüntüyü alırken çalışmakta olan `grep` sürecinin kendisini de. `pgrep` (Kısım 5.5) bu sürprizi önler.

3.4 Sütunları okumak

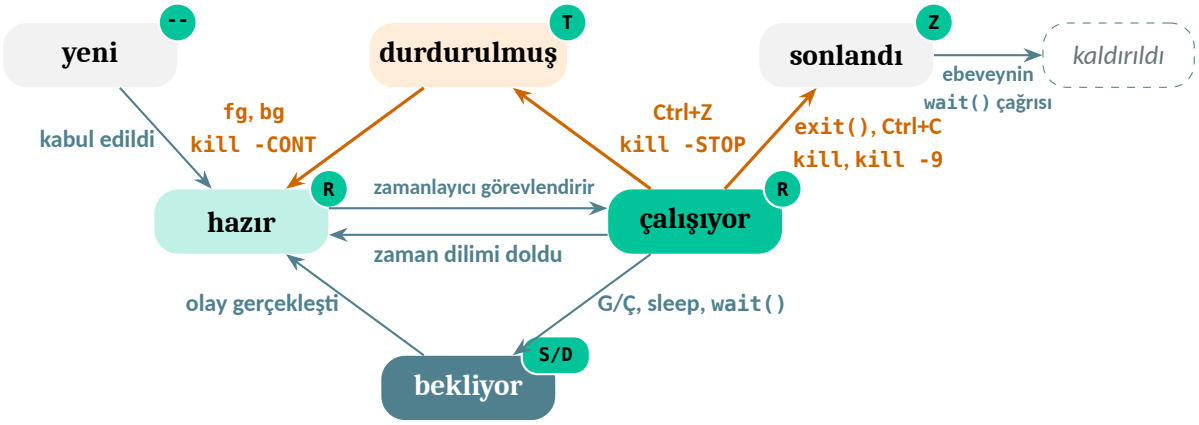
Sütun	Anlamı	Örnek (yukarıda)
`PID, PPID`	süreç kimliği ve ebeveynin süreç kimliği (3. hafta)	`1970`, `1913`
`UID, USER`	sürecin sahibi (11. hafta)	`student`
`STAT, S`	durum harfi ve bayraklar (Kısım 3.5)	`S`, `R+`, `Ss`
`%CPU, C`	işlemci zamanı bölü başlangıçtan beri geçen süre, %	`yes` için `99.0`
`%MEM`	yerleşik bellek bölü toplam RAM, %	`0.0`
`VSZ`	sanal bellek boyutu, KiB (6–7. haftalar)	`3132`
`RSS`	yerleşik küme boyutu (resident set size): RAM'deki kısım, KiB	`1884`
`TTY`	sürecin terminali; `?` = yok (daemon'lar)	`pts/0`
`STIME, START`	başlama zamanı	`16:58`
`TIME`	şimdiye dek kullanılan toplam işlemci zamanı	`0:02`
`CMD, COMMAND`	komut satırı (`[...]` = çekirdek iş parçacığı)	`sleep 300`

VSZ ve RSS 6–7. haftalara bağlanır: `yes` sürecinin 3132 KiB sanal adres alanı vardır, ama yalnızca 1884 KiB'ı RAM'de yerleşiktir — iş başındaki isteğe bağlı sayfalama. `ps`'in %CPU değeri sürecin *bütün ömrü* boyunca bir ortalamadır; `top` ise son birkaç saniyeyi ölçer. Çok iş parçacıklı bir süreç (4. hafta) %100'ün üstünde görünebilir: %200, iki işlemci çekirdeğinin tamamen dolu olduğu anlamına gelir.

3.5 ps içinde süreç durumları: STAT harfleri

3. hafta beş durum tanımlamıştı: *yeni* (new), *hazır* (ready), *çalışıyor* (running), *bekliyor* (waiting) ve *sonlandı* (terminated). Linux her süreç için `ps`'in STAT sütununda (`top`'ta S sütununda) bir durum harfi bildirir ve

ders kitabı modelinde olmayan bir durum ekler: *durdurulmuş* (stopped).



Şekil 2: Her durumun STAT harfiyle (nane rozetler) ve bu haftanın iş denetimi eylemleriyle (turuncu oklar) 3. haftanın durum diyagramı. Gri okları çekirdek ve zamanlayıcı izler.

Harf	Anlamı	3. hafta durumu	Bu notlarda nerede
R	çalışıyor ya da çalıştırılabilir: bir işlemcide ya da hazır kuyruğunda	çalışıyor *ya da* hazır	`yes > /dev/null`
S	kesilebilir uyku: bir olayı (zamanlayıcı, klavye, çocuk) bekliyor	bekliyor	`sleep`, `bash`
D	kesilemez uyku, genellikle disk G/Ç'sini bekliyor	bekliyor	seyrek ve kısa
T	Ctrl+Z ya da SIGSTOP ile durdurulmuş	(Linux eki)	Kısım 4
Z	zombi: sonlandı, çıkış durumu henüz toplanmadı	sonlandı	Kısım 7

Harften sonraki karakterler *bayraklardır*: `s` = oturum lideri (giriş kabuğunuz `Ss` gösterir), `+` = ön plan süreç grubunun üyesi (`ps`'in kendisi `R+` gösterir), `N` = düşük öncelik, yani pozitif nice değeri (`RN`, `SN`), `<` = yüksek öncelik, `l` = çok iş parçacıklı. Üç not:

- Linux *hazır* ile *çalışıyor* durumlarını ayırmaz: ikisi de R'dir. 2 çekirdekli bir makinede herhangi bir anda en çok 2 süreç gerçekten çalışır; öteki R süreçleri hazır kuyruğundadır.
- *Yeni* durumu yalnızca mikrosaniyeler sürer ve harfi yoktur.
- S durumundaki bir süreç de sonlandırılabilir: `sleep` bütün ömrünü S'de geçirir ve onu sürekli sonlandırıyoruz. Diyagram, okunaklı kalsın diye `kill` okunu yalnızca *çalışıyor* durumundan çiziyor.

3.6 `/proc/<pid>/status`: metin olarak PCB

3. haftadan beri Linux'un her PCB'nin bazı kısımlarını `/proc` altında metin dosyaları olarak gösterdiğini biliyoruz. `$!` en son arka plan işinin PID'sidir:

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2118
student@okan-lab:~/lab$ echo $!
2118
student@okan-lab:~/lab$ grep -E '^(Name|State|Pid|PPid|Uid|Threads|VmRSS)' /proc/$!/status
Name:   sleep
State:  S (sleeping)
Pid:    2118
PPid:   2061
Uid:    1013    1013    1013    1013
VmRSS:      2004 kB
Threads:    1
student@okan-lab:~/lab$ grep ctxt /proc/$!/status
voluntary_ctxt_switches:    1
nonvoluntary_ctxt_switches: 2
student@okan-lab:~/lab$ grep -E '^(Name|State|PPid)' /proc/$$/status
Name:   bash
State:  S (sleeping)
PPid:   2060
```

Alanlar 3. haftanın PCB'sine karşılık gelir: `State` (S, zamanlayıcısını bekliyor), `Pid` ve `PPid` (kimlik ve ebeveyn), `Uid` (sahibin kullanıcı numarası; laboratuvar makinemizde 1013, sizinkinde genellikle 1000, 11. hafta), `VmRSS` (bellek yönetimi bilgisi), `Threads` (4. hafta) ve bağlam değişimi sayaçları (hesap bilgisi: süreç beklemek için işlemciyi kendisi bıraktığında *gönüllü* (voluntary), zamanlayıcı işlemciyi elinden aldığında *gönülsüz* (nonvoluntary) bir geçiş olur). (`code/out_proc.txt` içindeki tam oturum `/proc/$!/cmdline` dosyasını ve `cwd` ile `exe` bağlantılarını da okur: komut satırı, çalışma dizini ve program dosyası.) Kendi kabuğumuz (`$$`) da uyuyor: klavyeyi bekliyor.

3.7 `top` ve `htop`: canlı görünüm

`top` süreçleri işlemci kullanımına göre sıralı gösterir ve birkaç saniyede bir yenilenir; çıkmak için q'ya basın. Bu notlara basabilmek için *toplu kipi* (batch mode, `-b`) tek bir yinelemeyle (`-n 1`) ve kullanıcı süzgeci `-u student` ile kullandık; bu sırada (yarım dakika önce & ile başlatılmış) bir `yes` ve bir `sleep` çalışıyordu:

```
student@okan-lab:~/lab$ cat /proc/loadavg
0.59 0.34 0.23 3/98 2262
student@okan-lab:~/lab$ nproc
2
student@okan-lab:~/lab$ top -b -n 1 -u student | head -15
top - 16:58:44 up 41 min,  0 user,  load average: 0.59, 0.34, 0.23
Tasks:  71 total,   2 running,  69 sleeping,   0 stopped,   0 zombie
%Cpu(s): 28.6 us, 23.8 sy,  0.0 ni, 42.9 id,  0.0 wa,  0.0 hi,  0.0 si,  4.8 st
MiB Mem :   8029.6 total,   7294.2 free,    598.8 used,    357.6 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.   7430.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 2260 student   20   0    3132   1876   1760 R 100.0   0.0   0:30.83 yes
 2204 student   20   0    8436   7528   3552 S   0.0   0.1   0:00.01 bash
 2261 student   20   0    3132   2004   1892 S   0.0   0.0   0:00.00 sleep
 2264 student   20   0    9380   5436   3308 R   0.0   0.1   0:00.00 top
 2265 student   20   0    3144   1956   1840 S   0.0   0.0   0:00.00 head
```

- **1. satır** (`uptime` çıktısının aynısı): şimdiki saat, açılıştan beri geçen süre, oturum açmış kullanıcı sayısı (laboratuvar makinemizde 0, çünkü konsolunda kimse oturum açmamış) ve son 1, 5 ve 15 dakikanın **yük ortalaması** (load average).

- **2. satır** makinenin bütün görevlerini durumlarına göre sayar: çalışan (R), uyuyan (S ve D), durdurulmuş (T), zombi (Z).
- **3. satır** *bütün makinenin* işlemci zamanını böler: `us` kullanıcı modu, `sy` çekirdek modu (ikili mod, 2. hafta), `ni` nice uygulanmış kullanıcı süreçleri, `id` boşta, `wa` G/Ç bekleme, `st` sanallaştırma konağının "çaldığı" zaman. 2 çekirdekten biri tamamen doluysa bu, makinenin yalnızca $1/2 = 50\%$'sidir.
- **4–5. satırlar**: MiB cinsinden bellek ve takas alanı (6–7. haftalar).
- **Süreç listesi**: PR önceliktir; normal süreçler için PR $= 20 +$ NI; VIRT/RES sanal ve yerleşik bellektir; S durumdur; %CPU son yenileme aralığında ölçülür. `yes` *bir* çekirdeğin %100'ünü kullanır.

`top` içindeki yararlı tuşlar: P işlemciye göre sırala, `M` belleğe göre sırala, u tek bir kullanıcıyı göster, k sonlandır (bir PID ve bir sinyal sorar), `r` renice, `1` her işlemciyi ayrı göster, q çık. `htop` daha kullanıcı dostu, renkli bir sürümdür: ok tuşlarıyla bir süreç seçin; F9 sinyal gönderir, F7/F8 nice değerini değiştirir, `F10` çıkar. Kurulu değilse `sudo apt install htop` ile kurun.

Çözümlü örnek: yük ortalamasını okumak

Yük ortalaması, çalışan ya da hazır (R durumu) süreçleri, artı kesilemez disk beklemesindeki (D) süreçleri sayar ve 1, 5 ve 15 dakika üzerinden ortalar. Yalnızca işlemci çekirdeği sayısı c'ye göre anlam taşır (`nproc` bu sayıyı yazdırır; laboratuvar makinemizde $c = 2$):

$$\begin{aligned} &\text{yük } 0.59: && 0.59/2 = 29.5\% \text{ kapasite kullanılıyor,} \\ &\text{yük } 2.00: && 2.00/2 = 100\%: \text{ her çekirdek dolu, kimse beklemiyor,} \\ &\text{yük } 3.00: && 3.00/2 = 150\%: \text{ ortalama } 3 - 2 = 1 \text{ süreç hazır kuyruğunda bekliyor.} \end{aligned}$$

1 dakikalık değer 15 dakikalık değerden büyükse yük artıyordur. Anlık görüntümüz `yes` başladıktan 30 s sonra alındı; bu yüzden 1 dakikalık ortalama (0,59) henüz 1'e tırmanmamıştı: ortalamalar anlık sayımlar değil, üstel olarak sönümlenen ortalamalardır. `/proc/loadavg` içinde dördüncü alan `3/98`, şu anda toplam 98 görevden 3'ünün çalıştırılabilir olduğu anlamına gelir; son sayı en son atanan PID'dir. 4. hafta terimleriyle: yük c'nin üstündeyse süreçler hazır kuyruğunda zaman geçirir, yani bekleme süreleri uzar. (Sayılar `code/check_numbers.py` ile denetlendi.)

4 İş Denetimi

İş (job), kabuğun başlattığı tek bir komut satırıdır: tek bir süreç ya da borularla bağlanmış birkaç süreç. Kabuk işlerini `[1]`, `[2]`, … diye numaralandırır. Klavyeye aynı anda yalnızca *bir* iş sahip olur: **ön plan** (foreground) işi. Ctrl+C ve Ctrl+Z tuşları sinyalleri (Kısım 5) yalnızca ön plan işine gönderir. **Arka planda** (background) istediğiniz kadar iş çalışabilir.

Ön planda (`komut`) kabuk bekler ve iş bitene dek istem göstermez; iş klavyeyi okur; Ctrl+C onu sonlandırır, Ctrl+Z durdurur. Kısa komutlar ve etkileşimli programlar (`nano`, `less`, `top`) için kullanın. **Arka planda** (`komut &`) kabuk `[iş] PID` ve istemi hemen yazdırır; tuşlar işe ulaşmaz (`kill %n` kullanın); iş klavyeden okumaya çalışırsa durdurulur (`Stopped (tty input)`); çıktısını yönlendirmezseniz (12. hafta) yine terminalde görünür. Uzun, etkileşimsiz işler için kullanın: derleme, kopyalama, indirme.

4.1 Bir işi Ctrl+Z ile durdurmak, bg ile sürdürmek

```
student@okan-lab:~/lab$ sleep 100
^Z
[1]+  Stopped                 sleep 100
student@okan-lab:~/lab$ jobs
[1]+  Stopped                 sleep 100
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
 2345 Ss   -bash
 2401 T    sleep 100
 2402 R+   ps -o pid,stat,cmd
student@okan-lab:~/lab$ bg
[1]+ sleep 100 &
student@okan-lab:~/lab$ jobs -l
[1]+  2401 Running                 sleep 100 &
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
 2345 Ss   -bash
 2401 S    sleep 100
 2403 R+   ps -o pid,stat,cmd
```

1. `sleep 100` ön planda çalışıyor: istem yok. **Ctrl+Z**'ye basmak (ekranda ^Z olarak görünür) onu durdurur; kabuk `[1]+ Stopped` bildirir ve istemi geri verir.
2. `jobs` bu kabuğun işlerini listeler; `ps` `T` (durdurulmuş) durumunu gösterir. Durdurulmuş bir süreç belleğini ve açık dosyalarını korur ama işlemci zamanı almaz: 3. hafta terimleriyle ne hazır ne de bekliyor durumundadır; Linux'un ek duruma bu yüzden ihtiyacı vardır.
3. `bg` geçerli işin *arka planda* sürmesini sağlar; kabuk komutu sonuna bir `&` ekleyerek yazdırır. `jobs -l` PID'leri ekler. `sleep` yeniden `S` durumundadır: takılmadı, yine zamanlayıcısını bekliyor.

4.2 & ile başlatmak, fg, Ctrl+C ve kill %n

```
student@okan-lab:~/lab$ sleep 200 &
[2] 2404
student@okan-lab:~/lab$ yes > /dev/null &
[3] 2405
student@okan-lab:~/lab$ jobs
[1]   Running                 sleep 100 &
[2]-  Running                 sleep 200 &
[3]+  Running                 yes > /dev/null &
student@okan-lab:~/lab$ fg %3
yes > /dev/null
^C
student@okan-lab:~/lab$ jobs
[1]-  Running                 sleep 100 &
[2]+  Running                 sleep 200 &
student@okan-lab:~/lab$ kill %1 %2
student@okan-lab:~/lab$ jobs
[1]-  Terminated              sleep 100
[2]+  Terminated              sleep 200
student@okan-lab:~/lab$ jobs
```

- `[3] 2405`: 3 numaralı iş, PID 2405. `jobs` listesinde `+` *geçerli* işi (argümansız `fg` ve `bg` için varsayılan), `-` ise bir öncekini gösterir.
- `fg %3` `yes`'i ön plana getirir (kabuk komut satırını yazdırır); **Ctrl+C** (^C) onu sonlandırır.

- `kill %1 %2` 1 ve 2 numaralı işlere SIGTERM gönderir. Kabuk `Terminated` iletisini bir kez, bir sonraki istemden hemen önce bildirir; ikinci `jobs` hiçbir şey yazdırmaz.

4.3 İşleri ve süreçleri adlandırmak; `$$` ve `$!`

Yazdığınız	Anlamı	Örnek
`%1, %2, …`	bu kabuğun n numaralı işi	`fg %2, kill %1`
`%+` ya da `%%`	geçerli iş (+ ile işaretli)	`fg`
`%-`	bir önceki iş (- ile işaretli)	`bg %-`
`%sleep`	komutu `sleep` ile başlayan iş	`kill %sleep`
`1970`	bir PID: her terminalde ve her araçla geçerli	`kill 1970`
`$$`	geçerli kabuğun PID'si	`echo $$`
`$!`	en son arka plan işinin PID'si	`kill $!`

İş numaraları yalnızca işleri başlatan kabuğun içinde vardır: ikinci bir terminalin kendi `[1]`'i olur. PID'ler ise geneldir. **Dikkat:** `kill 1` (% olmadan) 1 numaralı işi değil, PID 1'i belirtir — Kısım 9'daki sık hatalara bakın. `$$` ve `$!` değişkenleri 14. haftanın betiklerinde işe yarayacak.

5 Sinyaller

5.1 Sinyal nedir?

Sinyal (signal), çekirdeğin bir sürece ilettiği küçük, numaralı bir bildirimdir. 2. haftadaki donanım kesmesinin yazılımdaki karşılığıdır: bir kesme işlemcinin yaptığı işi bırakıp bir işleyici çalıştırmasına yol açar; bir sinyal ise tek bir *sürecin* yaptığı işi bırakıp tepki vermesine yol açar. Sinyalleri kullanıcılar (tuşlar, `kill` komutu), başka süreçler ve çekirdeğin kendisi gönderir (örneğin bir çocuk sonlandığında SIGCHLD, geçersiz bir bellek erişiminden sonra SIGSEGV).

Çekirdek sinyali *bekleyen* (pending) olarak işaretler ve hedef sürece iletir; süreç üç biçimde tepki verir: o sinyalin *varsayılan eylemi* (sonlan, dur, sürdür ya da yok say), kendi *işleyici* (handler) fonksiyonu (sinyali *yakalamak*) ya da sinyali yok saymak. İki sinyal özeldir: **SIGKILL (9)** ve **SIGSTOP (19)** ne yakalanabilir ne de yok sayılabilir — çekirdek tek başına davranır. Sıradan kullanıcılar yalnızca kendi süreçlerine sinyal gönderebilir; `root` herkesinkine gönderebilir (bir koruma kuralı, 11. hafta).

> **Tarihsel not: neden "kill" ve "hang up"?**
> İlk UNIX'te sinyal gönderen komuta `kill` (öldür) adı verildi, çünkü sinyallerin çoğunun varsayılan eylemi süreci sonlandırmaktır; `kill -STOP` ya da `kill -CONT` hiçbir şeyi öldürmese de ad kaldı. SIGHUP ("hang up", telefonu kapatmak) ise terminallerin telefon hatları ve modemlerle bağlandığı dönemden gelir: hat kapandığında o oturumun süreçleri SIGHUP alırdı. Bugün bir terminal penceresini ya da bir SSH bağlantısını kapatmak aynı şeyi yapar.

5.2 Sinyal tablosu

`kill -l`, Linux'un 64 sinyal numarasının tümünü listeler (aşağıda ilk satırlar; tam liste `code/out_signals.txt` dosyasında). `kill -l 15` bir numarayı ada, `kill -l KILL` bir adı numaraya çevirir.

```
student@okan-lab:~/lab$ kill -l
 1) SIGHUP   2) SIGINT   3) SIGQUIT  4) SIGILL   5) SIGTRAP
 6) SIGABRT  7) SIGBUS   8) SIGFPE   9) SIGKILL 10) SIGUSR1
11) SIGSEGV 12) SIGUSR2 13) SIGPIPE 14) SIGALRM 15) SIGTERM
```

No.	Ad	Gönderen	Varsayılan eylem	Yakalanır mı?
1	SIGHUP	terminalin ya da SSH oturumunun kapanması	sonlan	evet
2	SIGINT	Ctrl+C	sonlan	evet
3	SIGQUIT	Ctrl+\	sonlan + bellek dökümü	evet
9	SIGKILL	`kill -9`: son çare	hemen sonlan	**hayır**
11	SIGSEGV	geçersiz bir bellek erişiminden sonra çekirdek (7. hafta)	sonlan + bellek dökümü	evet
15	SIGTERM	`kill` (varsayılan), `pkill`, `killall`, sistemin kapatılması	sonlan	evet
17	SIGCHLD	bir çocuk sonlandığında ya da durduğunda çekirdek	yok say	evet
18	SIGCONT	`fg`, `bg`, `kill -CONT`	durmuşsa sürdür	evet
19	SIGSTOP	`kill -STOP`	dur	**hayır**
20	SIGTSTP	Ctrl+Z	dur	evet

Numaralar x86 ve ARM üzerindeki Linux'unkilerdir; bazıları başka UNIX sistemlerinde farklıdır, bu yüzden adlar numaralardan daha güvenlidir (her yerde aynı olan 9 ve 15 dışında). Adı ön ekle ya da ön eksiz yazabilirsiniz: `kill -TERM`, `kill -SIGTERM` ve `kill -15` eşdeğerdir. ("Bellek dökümü", İngilizce *core dump*: sürecin belleğinin hata ayıklama için bir dosyaya yazılması.)

5.3 SIGTERM, SIGKILL, SIGINT ve çıkış durumu

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2541
student@okan-lab:~/lab$ kill $!
student@okan-lab:~/lab$ wait $!
[1]+  Terminated              sleep 300
student@okan-lab:~/lab$ echo $?
143
student@okan-lab:~/lab$ sleep 300 &
[1] 2542
student@okan-lab:~/lab$ kill -9 $!
student@okan-lab:~/lab$ wait $!
[1]+  Killed                  sleep 300
```

```
student@okan-lab:~/lab$ echo $?
137
```

Yerleşik `wait $!` komutu arka plan işinin bitmesini bekler ve kabuğun işin *nasıl* bittiğini bildirmesini sağlar: `Terminated` (SIGTERM), `Killed` (SIGKILL) ya da `Interrupt` (SIGINT: tam çıktı dosyasındaki `kill -INT` ile yapılan üçüncü deney `Interrupt` yazdırır). Ardından son komutun çıkış durumu (exit status) olan `$?` sinyali kodlar:

Sinyalle öldürülen bir sürecin çıkış durumu

Bir süreç n numaralı sinyalle sonlandırılırsa kabuk onun çıkış durumunu $128 + n$ yapar:

$$\text{SIGTERM: } 128 + 15 = 143, \qquad \text{SIGKILL: } 128 + 9 = 137, \qquad \text{SIGINT: } 128 + 2 = 130.$$

Normal bir çıkış 0 ile 127 arasında bir durum verir (başarı için genellikle 0). Tersine, $s > 128$ olan bir durum "$s - 128$ numaralı sinyalle öldürüldü" demektir.

5.4 Duraklatmak ve sürdürmek: SIGSTOP ve SIGCONT

```
student@okan-lab:~/lab$ sleep 300 &
[1] 2544
student@okan-lab:~/lab$ kill -STOP $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
 2544 T    sleep 300

[1]+  Stopped                 sleep 300
student@okan-lab:~/lab$ kill -CONT $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd -p $!
  PID STAT CMD
 2544 S    sleep 300
student@okan-lab:~/lab$ kill $!
```

`kill -STOP sleep`'i `T` durumuna sokar — Ctrl+Z'nin ürettiği durumun ta kendisi; kabuğun işi `Stopped` olarak bildirmesinin nedeni budur. `kill -CONT` sürmesini sağlar (yeniden `S`). `fg` ve `bg`'nin içeride yaptığı da budur: SIGCONT gönderirler.

5.5 Süreçleri adlarıyla seçmek: `pgrep`, `pkill`, `killall`

```
student@okan-lab:~/lab$ sleep 401 &
[2] 2548
[1]   Terminated              sleep 300
student@okan-lab:~/lab$ sleep 402 &
[3] 2549
student@okan-lab:~/lab$ yes > /dev/null &
[4] 2550
student@okan-lab:~/lab$ pgrep -u student -l sleep
2548 sleep
2549 sleep
student@okan-lab:~/lab$ pkill -u student sleep
[2]   Terminated              sleep 401
[3]-  Terminated              sleep 402
student@okan-lab:~/lab$ killall -u student yes
[4]+  Terminated              yes > /dev/null
student@okan-lab:~/lab$ jobs
```

```
student@okan-lab:~/lab$ pgrep -u student -l sleep
student@okan-lab:~/lab$ echo $?
1
```

`pgrep`, adı bir kalıpla eşleşen süreçlerin PID'lerini yazdırır (`-l` adı ekler, `-c` sayar); `pkill` onlara bir sinyal gönderir (varsayılan SIGTERM); `killall` aynısını tam bir program adıyla yapar. Son `pgrep` hiçbir şey bulamaz ve çıkış durumu 1'dir. **Her zaman `-u student` ekleyin** (kendi kullanıcı adınız): bu seçenek olmadan araçlar başka kullanıcıların süreçlerine de sinyal göndermeyi dener; bu "Operation not permitted" ile başarısız olur — root olarak çalıştırılırsa ise onları gerçekten öldürür. `pkill`'i kullanmadan önce kalıbı `pgrep` ile deneyin: `pkill -u student bash` bütün terminallerinizi kapatırdı.

5.6 Sinyalleri yakalamak: `catch.c`

Bir program sinyallerin çoğu için kendi *işleyicisini* kurabilir. `code/catch.c` programı SIGINT ve SIGTERM'i yakalar, ayrıca SIGKILL için bir işleyici kurmayı dener — ve başaramaz. Listede programın özü görülüyor; tam dosya `#include` satırlarını, `struct sigaction` hazırlığını ve başlangıç iletisini ekler:

```c
static volatile sig_atomic_t got = 0;
static void handler(int sig) { got = sig; }  /* minicik! */

sa.sa_handler = handler;
sigaction(SIGINT,  &sa, NULL);   /* Ctrl+C, 2: yakalanır */
sigaction(SIGTERM, &sa, NULL);   /* kill,  15: yakalanır */
if (sigaction(SIGKILL, &sa, NULL) < 0)  /* 9: reddedilir */
    perror("catch: sigaction(SIGKILL)");

for (;;) {
    pause();                     /* sinyal gelene dek uyu */
    if (got == SIGINT)
        printf("... ignoring it, still running\n");
    if (got == SIGTERM) {
        printf("... cleaning up, bye\n");
        return 0;
    }
}
```

`gcc -Wall -O2` ile derlendi (uyarı yok) ve `~/lab/src` içinde çalıştırıldı:

```
student@okan-lab:~/lab$ cd src
student@okan-lab:~/lab/src$ gcc -Wall -O2 -o catch catch.c
student@okan-lab:~/lab/src$ ./catch
catch: sigaction(SIGKILL): Invalid argument
catch: PID 2694, waiting for signals
^Ccatch: got SIGINT (2) -- ignoring it, still running
^Ccatch: got SIGINT (2) -- ignoring it, still running
^Z
[1]+  Stopped                 ./catch
student@okan-lab:~/lab/src$ bg
[1]+ ./catch &
student@okan-lab:~/lab/src$ kill -INT %1
student@okan-lab:~/lab/src$ catch: got SIGINT (2) -- ignoring it, still running
kill -TERM %1
student@okan-lab:~/lab/src$ catch: got SIGTERM (15) -- cleaning up, bye
jobs
[1]+  Done                    ./catch
```

```
student@okan-lab:~/lab/src$ ./catch &
[1] 2695
student@okan-lab:~/lab/src$ catch: sigaction(SIGKILL): Invalid argument
catch: PID 2695, waiting for signals
kill -9 %1
student@okan-lab:~/lab/src$ jobs
[1]+  Killed                  ./catch
```

SIGKILL işleyicisinin kurulumu başarısız olur (`Invalid argument`): çekirdek onu reddeder. Ctrl+C ve `kill -INT` artık programı sonlandırmaz — işleyici çalışır ve program devam eder. Ctrl+Z onu hâlâ durdurur (SIGTSTP yakalanmıyor) ve `bg` sürdürür. SIGTERM programın temizlik yapıp normal biçimde çıkmasını sağlar (`jobs` `Terminated` değil, `Done` bildirir); `kill -9` ise her zaman işe yarar (`Killed`). Bir arka plan işinin yazdırdığı iletiler imleç neredeyse oraya — istemin hemen arkasına — basılır; bu yüzden yazdığınız sonraki komut yeni bir satırda başlar. Tuhaf görünür ama normaldir.

5.7 Bir süreci güvenle durdurmak

Önce rica et, en son zorla

1. Ön plan işi: Ctrl+C'ye basın (SIGINT).
2. Değilse: `kill PID` (SIGTERM) ve 5–10 saniye bekleyin.
3. `ps -p PID` ile hâlâ orada olup olmadığına ve hangi durumda olduğuna bakın.
4. Ancak ondan sonra: `kill -9 PID` (SIGKILL).

`kill -9` hakkında uyarı. SIGKILL programa açık belgeleri kaydetme, tamponları boşaltma ya da geçici ve kilit dosyalarını silme fırsatı vermez; sonrasında bir veritabanının onarılması gerekebilir ve programın çocuklarına haber verilmez. Bazen `kill -9` hiç işe yaramaz: bir zombi (`Z`) zaten ölüdür (Kısım 7); `D` durumundaki bir süreç sinyali ancak disk G/Ç'si bittiğinde alır; başka bir kullanıcının sürecine ise sinyal gönderilemez ("Operation not permitted").

5.8 SIGHUP ve nohup

Bir terminali (ya da bir SSH bağlantısını) kapattığınızda kabuğun işleri SIGHUP alır ve varsayılan olarak sonlanır. `nohup komut`, SIGHUP'ı yok sayan bir komut başlatır. Bu oturumda terminalin kapanmasını `kill -HUP` ile taklit ediyoruz:

```
student@okan-lab:~/lab$ echo "my shell is PID $$"
my shell is PID 2775
student@okan-lab:~/lab$ sleep 500 &
[1] 2831
student@okan-lab:~/lab$ echo "last background job is PID $!"
last background job is PID 2831
student@okan-lab:~/lab$ nohup sleep 600 &
[2] 2832
student@okan-lab:~/lab$ nohup: ignoring input and appending output to 'nohup.out'
nohup sleep 700 > /dev/null 2>&1 &
[3] 2834
student@okan-lab:~/lab$ ls
README.txt  backup  data  docs  logs  nohup.out  photos  projects  scripts  src
student@okan-lab:~/lab$ jobs -l
[1]   2831 Running                 sleep 500 &
```

```
[2]-  2832 Running                 nohup sleep 600 &
[3]+  2834 Running                 nohup sleep 700 > /dev/null 2>&1 &
student@okan-lab:~/lab$ kill -HUP %1 %2 %3
student@okan-lab:~/lab$ jobs -l
[1]   2831 Hangup                  sleep 500
[2]-  2832 Running                 nohup sleep 600 &
[3]+  2834 Running                 nohup sleep 700 > /dev/null 2>&1 &
student@okan-lab:~/lab$ kill %2 %3
```

Yalın `sleep 500` sona erdi (Hangup), iki `nohup` işi çalışmayı sürdürdü. Bir `nohup` komutunun çıktısı terminale gidiyorsa `nohup` onu `nohup.out` dosyasına yönlendirir (ve bunu söyler: ileti istemden sonra çıktı, çünkü arka plandaki `nohup` onu bir an sonra yazdırdı). Temiz biçim şudur:

```
nohup long_command > run.log 2>&1 &
```

Bu biçim 12. haftanın yönlendirmelerini kullanır. Tipik kullanım: SSH üzerinden uzak bir sunucuda uzun bir iş başlatın, oturumu kapatın ve `run.log` dosyasına daha sonra bakın.

6 Öncelikler: `nice` ve `renice`

5. haftada öncelikli zamanlamayla ve işlemciyi çalıştırılabilir süreçler arasında bir *ağırlıkla* orantılı olarak paylaştıran Linux zamanlayıcısıyla tanışmıştık. Ağırlık, -20 ile $+19$ arasında bir tamsayı olan **nice değerinden** (nice value) gelir (40 değer, varsayılan 0). 5. haftadaki gibi *küçük* sayı *yüksek* öncelik demektir: nice değeri yüksek bir süreç ötekilere karşı “naziktir” (nice). `top` önceliği PR $= 20 +$ NI olarak gösterir.

```
student@okan-lab:~/lab$ nice
0
student@okan-lab:~/lab$ nice -n 10 yes > /dev/null &
[1] 2971
student@okan-lab:~/lab$ ps -o pid,ni,stat,cmd
  PID  NI STAT CMD
 2914   0 Ss   -bash
 2971  10 RN   yes
 2972   0 R+   ps -o pid,ni,stat,cmd
student@okan-lab:~/lab$ renice -n 15 -p $!
2971 (process ID) old priority 10, new priority 15
student@okan-lab:~/lab$ renice -n 5 -p $!
renice: failed to set priority for 2971 (process ID): Permission denied
student@okan-lab:~/lab$ nice -n -5 sleep 1
nice: cannot set niceness: Permission denied
student@okan-lab:~/lab$ kill %1
student@okan-lab:~/lab$ taskset -c 1 yes > /dev/null &
[2] 2976
[1]   Terminated              nice -n 10 yes > /dev/null
student@okan-lab:~/lab$ taskset -c 1 nice -n 10 yes > /dev/null &
[3] 2977
student@okan-lab:~/lab$ ps -o pid,ni,stat,%cpu,cmd
  PID  NI STAT %CPU CMD
 2914   0 Ss    0.0 -bash
 2976   0 R    89.3 yes
 2977  10 RN    9.6 yes
 2978   0 R+    0.0 ps -o pid,ni,stat,%cpu,cmd
student@okan-lab:~/lab$ kill %2 %3
```

- Argümansız `nice` geçerli nice değerini yazdırır: 0. `nice -n 10 komut` bir komutu nice değeri 10 ile başlatır; `ps` `NI` `10` ve `N` bayrağını (RN) gösterir.

- `renice -n 15 -p PID` çalışan bir sürecin değerini değiştirir. `renice` iletisinde buna "priority" der; kastedilen nice değeridir. Değeri yükseltmek ($10 \rightarrow 15$) serbesttir; yeniden düşürmek ($15 \rightarrow 5$) sıradan kullanıcılar için `Permission denied` ile sonuçlanır — yoksa herkes -20 ayarlardı. `nice -n -5` bir uyarı yazdırır ve komutu nice 0 ile çalıştırır.
- Son deney, 5. haftanın ölçümünü `yes` ile tekrarlar: işlemci yoğun iki süreç `taskset -c 1` ile aynı çekirdeğe bağlanır (bağlanmasalardı her biri kendi çekirdeğini alır ve ikisi de yaklaşık %100 gösterirdi).

Çözümlü örnek: işlemci paylarını öngörmek

Çekirdeğin nice 0 için ağırlığı 1024, nice 10 için 110'dur (her nice adımı ağırlığı yaklaşık 1.25 katı değiştirir ve $1.25^{10} \approx 9.31$). Tek çekirdekteki işlemci yoğun iki süreç şu payları alır:

$$\frac{1024}{1024+110} \approx 90.3\,\%, \qquad \frac{110}{1024+110} \approx 9.7\,\%, \qquad \text{oran } \frac{1024}{110} \approx 9.31.$$

Biz %89,3 ve %9,6 ölçtük; oran $89.3/9.6 \approx 9.30$. (İki değerin toplamı tam olarak %100 etmez, çünkü `ps` her sürecin ömrü boyunca ortalama alır ve iki süreç tam aynı anda başlamadı. Sayılar `code/check_numbers.py` ile denetlendi.)

`root` olarak `sudo renice -n -5 -p PID` işe yarardı (11. hafta) — ama paylaşılan bir makinede kendi süreçlerinize herkesinkinden yüksek öncelik vermeyin.

7 Zombiler ve Yetimler, Görünür Hâlde

3. hafta **zombiyi** (zombie), sonlanmış ama ebeveyni çıkış durumunu henüz `wait()` ile toplamamış bir süreç olarak tanımlamıştı. Çekirdek onun belleğini zaten serbest bırakmış ve dosyalarını kapatmıştır; geriye yalnızca PID'yi ve çıkış durumunu tutan süreç tablosu girdisi kalır. 3. haftada bir zombiyi bir kabuk hilesiyle üretmiştik; şimdi bunu bilerek yapan en küçük C programını yazıyoruz, `code/zombie.c`:

```c
pid_t pid = fork();                /* bir süreç ikiye ayrılır */

if (pid == 0) {                    /* ---- çocuk ---- */
    printf("child  %d: exiting now with status 7\n", getpid());
    exit(7);                       /* sonlandı -> toplanana dek zombi */
}
/* ---- ebeveyn: secs saniye boyunca wait() ÇAĞIRMAZ ---- */
printf("parent %d: child is %d, sleeping %d s without wait()\n",
       getpid(), pid, secs);
sleep(secs);                       /* bu sırada çocuk bir zombidir */

waitpid(pid, &status, 0);          /* çıkış durumunu topla (reap) */
printf("parent %d: reaped child %d, exit status %d\n",
       getpid(), pid, WEXITSTATUS(status));
```

Listede programın özü görülüyor; tam dosya `#include` satırlarını ekler, `secs` değerini komut satırından okur (varsayılan 30) ve başarısız bir `fork()` durumunu denetler. Çocuk hemen 7 durumuyla çıkar; ebeveyn `secs` saniye boyunca `wait()` çağır*madan* uyur, sonra çocuğu `waitpid()` ile toplar (reap). `zombie.c`'yi `~/lab/src` içine kopyalayıp çalıştırdık:

```
student@okan-lab:~/lab$ cd src
```

```
student@okan-lab:~/lab/src$ gcc -Wall -O2 -o zombie zombie.c
student@okan-lab:~/lab/src$ ./zombie 25 &
[1] 3119
student@okan-lab:~/lab/src$ parent 3119: child is 3120, sleeping 25 s without wait()
child  3120: exiting now with status 7
ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3119  3057 S    ./zombie 25
 3120  3119 Z    [zombie] <defunct>
 3121  3057 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab/src$ grep State /proc/$(pgrep -u student -n zombie)/status
State:  Z (zombie)
student@okan-lab:~/lab/src$ kill -9 $(pgrep -u student -n zombie)
student@okan-lab:~/lab/src$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3119  3057 S    ./zombie 25
 3120  3119 Z    [zombie] <defunct>
 3125  3057 R+   ps -o pid,ppid,stat,cmd
student@okan-lab:~/lab/src$ wait
parent 3119: reaped child 3120, exit status 7
[1]+  Done                    ./zombie 25
student@okan-lab:~/lab/src$ ps -o pid,ppid,stat,cmd
  PID  PPID STAT CMD
 3057  3056 Ss   -bash
 3128  3057 R+   ps -o pid,ppid,stat,cmd
```

- PID 3120, yani çocuk, Z durumundadır; adı köşeli parantez içinde gösterilir ve `<defunct>` olarak işaretlenir; ebeveyni 3119'dur. `/proc` da aynı fikirdedir: `State: Z (zombie)`. (`pgrep -n` eşleşen süreçlerin *en yenisini* seçer; o da çocuktur.)
- Zombiye `kill -9` göndermek *hiçbir şey* yapmaz: zaten ölmüş olanı öldüremezsiniz.
- Kabuğun `wait` komutu arka plan işi `./zombie`'yi bekler; 25 saniye sonra ebeveyn `waitpid()` çağırır, 7 çıkış durumunu alır ve zombi listeden kaybolur.

Birkaç zombi zararsızdır. Ancak bir döngü içinde çocuk oluşturup hiçbirini beklemeyen hatalı bir ebeveyn süreç tablosunu doldurabilir: PID'ler sınırlıdır (makinemizde `/proc/sys/kernel/pid_max` 32768'dir, yani en büyük PID 32767) ve tablo dolunca yeni süreç oluşturulamaz. Çare *ebeveyni* düzeltmek ya da sonlandırmaktır: ebeveyn sonlanınca zombi bir **yetime** (orphan) dönüşür, PID 1 (ya da bir "subreaper" süreci) tarafından evlat edinilir ve bu yeni ebeveyn onu hemen toplar (3. hafta). Çekirdek ebeveynlere SIGCHLD sinyaliyle yardım eder; bu sinyal bir çocuk sonlandığında gönderilir ve iyi yazılmış bir ebeveyn buna `wait()` çağırarak tepki verir.

8 Komut Başvuru Tablosu

İnceleme		İş denetimi	
`ps / ps -f`	bu terminal (kısa/tam)	`cmd &`	arka planda başlat
`ps -ef / ps aux`	bütün süreçler	`Ctrl+C / Ctrl+Z`	ön plan işine SIGINT / SIGTSTP
`ps -u student`	bir kullanıcının süreçleri	`jobs / jobs -l`	işleri listele (PID'lerle)
`ps -o pid,stat,cmd`	sütunları seç	`fg %n / bg %n`	ön plana getir / arka planda sürdür
`ps -p PID`	tek bir süreç	`wait [%n]`	bir arka plan işini bekle
`ps --sort =-%cpu`	en yüksek işlemci önce		
İnceleme (devam)		**Sinyaller**	
`pstree -p [PID]`	PID'leriyle süreç ağacı	`kill PID / kill %n`	SIGTERM (15)
`pgrep -u student -l name`	PID'ler ve adlar (`-c` sayı, `-n` en yeni)	`kill -INT / -HUP / -9`	SIGINT, SIGHUP, SIGKILL
`top / htop`	canlı görünüm (q ile çıkılır)	`kill -STOP / -CONT`	duraklat / sürdür
`uptime / nproc`	yük ortalaması / çekirdek sayısı	`kill -l [n]`	sinyalleri listele / numarayı ada çevir
`cat /proc/PID/status`	metin olarak PCB	`pkill -u student pat`	ad kalıbıyla sinyal
`echo $$ / echo $!`	kabuğun / son arka plan işinin PID'si	`killall -u student name`	tam adla sinyal
Öncelik		**Oturum kapanınca sürmek**	
`nice -n 10 cmd`	nice değeri 10 ile başlat	`nohup cmd > log 2>&1 &`	SIGHUP'ı yok say, çıktı `log`'a
`renice -n 15 -p PID`	değiştir (kullanıcı: yalnızca yukarı)	`taskset -c 1 cmd`	yalnızca 1. çekirdekte çalıştır (deneyler)

9 Sık Yapılan Hatalar ve Anlamları

`-bash: kill: (1) - Operation not permitted` `kill 1` yazdınız ama 1 numaralı işi kastettiniz. `%` olmadan sayı bir PID'dir ve PID 1 root'a aittir. `kill %1` kullanın. (Gerçek çıktı, Laboratuvar Föyü çözümü B4.)

`-bash: kill: %6: no such job` *Bu* kabukta 6 numaralı iş yok. `jobs` ile bakın; başka bir terminalin işlerine PID ile başvurulmalıdır.

`renice: failed to set priority for ... Permission denied` Sıradan bir kullanıcı nice değerini düşürmeye çalıştı. Yalnızca yükseltmeye izin vardır.

`nice: cannot set niceness: Permission denied` Sıradan kullanıcı olarak negatif bir nice değeri. Komut yine de nice 0 ile çalışır.

`pkill: killing pid ... failed: Operation not permitted` Kalıp başka bir kullanıcının süreciyle de eşleşti. Kullanıcı adınızla `-u` ekleyin.

İstem geri gelmiyor Bir ön plan işi çalışıyor (& koymayı unuttunuz). İşi bitirmek için Ctrl+C'ye, ya da arka

planda sürdürmek için Ctrl+Z'ye ve ardından bg'ye basın.

`[1]+ Stopped (tty input)` Bir arka plan işi klavyeden okumak istedi (`nano &` ya da `cat &` deneyin). Onu `fg` ile ön plana getirin.

`There are stopped jobs.` Durdurulmuş işler varken `exit` yazdınız. Onları sürdürün ya da sonlandırın (`jobs`, `fg`, `kill %n`); `exit`'i ikinci kez yazmak yine de çıkar ve durdurulmuş işler sonlandırılır.

Bir `Z <defunct>` satırı `kill -9`'dan sağ çıkıyor Zombi zaten ölüdür. Onu yalnızca ebeveyninin bir `wait()` çağrısı kaldırır; ebeveyni sonlandırın ya da düzeltin.

`grep` kendi sonucunda görünüyor `ps aux | grep sleep` gibi bir boru hattı `grep` sürecinin kendisini de listeler. Bunun yerine `pgrep -l sleep` kullanın.

Türkçe yerel ayarlı bir sistemde bu iletilerin bazıları Türkçe görünebilir; anlamları aynıdır.

10 Özet

Sekiz cümlede 13. hafta

1. Her komut, kabuğun `fork()` ve `exec()` ile oluşturduğu bir süreçtir; ön planda kabuk `wait()` çağırır, `&` ile çağırmaz.
2. `ps` (`-f`, `-ef`, `aux`, `-o`) ve `pstree` anlık görüntü alır; `top`/`htop` canlı bir görünüm verir; `/proc/<pid>/status` PCB'yi gösterir.
3. STAT harfleri: R = hazır ya da çalışıyor, S/D = bekliyor, T = durdurulmuş (Linux eki), Z = sonlandı ama henüz toplanmadı.
4. Yük ortalaması işlemci isteyen süreçleri sayar; onu işlemci çekirdeği sayısıyla karşılaştırın.
5. Ctrl+Z ön plan işini durdurur, `bg` ve `fg` onu sürdürür, `jobs` işleri listeler; `%n` bir işi, yalın bir sayı bir PID'yi adlandırır.
6. `kill` sinyal gönderir (varsayılan SIGTERM, 15); Ctrl+C SIGINT (2) gönderir; SIGKILL (9) ve SIGSTOP (19) yakalanamaz; n sinyaliyle öldürülen bir sürecin çıkış durumu $128 + n$'dir.
7. `nohup` bir işi SIGHUP'tan korur; `nice`/`renice` kullanıcıların yalnızca yükseltebildiği $-20 \ldots 19$ nice değerini ayarlar; nice 0'a karşı nice 10 işlemcinin yaklaşık $1/9.3$'ünü alır.
8. Bir zombi öldürülemez; onu yalnızca ebeveyninin `wait()` çağrısı (ya da ebeveynin ölümü ve PID 1'in onu evlat edinmesi) kaldırır.

11 Sözlük

Arka plan işi (background job) `&` ile başlatılan iş; kabuk onu beklemez.

Durdurulmuş (stopped, T) SIGTSTP ya da SIGSTOP ile duraklatılmış bir sürecin durumu; SIGCONT ile sürdürülür.

İş (job) Kabuğun başlattığı bir komut satırı (bir ya da daha çok süreç); `[1]`, `[2]`, … diye numaralanır.

Nice değeri (nice value) Bir sürecin işlemci ağırlığını belirleyen -20 ile 19 arasında sayı; yüksek değer "daha nazik" (daha düşük öncelik) demektir.

Ön plan işi (foreground job) Klavye girdisini ve Ctrl+C ile Ctrl+Z sinyallerini alan tek iş.

Sinyal (signal) Çekirdeğin bir sürece ilettiği numaralı yazılımsal bildirim.

SIGKILL / SIGTERM 9 numaralı sinyal (yakalanamaz, hemen sonlandırır) / 15 numaralı sinyal (sonlanmak için kibar bir rica).

Yetim (orphan) Ebeveyni sonlanmış süreç; PID 1 tarafından evlat edinilir.

Yük ortalaması (load average) Çalışan, hazır ya da disk beklemesindeki süreçlerin 1, 5 ve 15 dakika üzerinden ortalama sayısı.

Zombi (zombie, Z) Çıkış durumu ebeveyni tarafından henüz toplanmamış, sonlanmış süreç.

12 Kendiniz Deneyin

Kendiniz deneyin: kendi makinenizde süreçler

Linux (laboratuvar bilgisayarları, WSL ya da bir sanal makine), `~/lab` içinde; iki programı `gcc -Wall -O2 -o zombie zombie.c` ve `gcc -Wall -O2 -o catch catch.c` ile derleyin.

1. Bir terminalde `top`'u başlatın, başka bir terminalde `yes > /dev/null &` komutunu üç kez çalıştırın. Bir dakika boyunca yük ortalamasının tırmanışını izleyin. Hangi değere yaklaşıyor?
2. `nano notes.txt` çalıştırın, Ctrl+Z'ye basın, `ls` çalıştırın, sonra `fg`. Metniniz hâlâ orada.
3. `catch.c`'yi SIGTSTP'yi de yakalayacak biçimde değiştirin. Şimdi Ctrl+Z'ye basınca ne olur?
4. `./zombie 60 &` çalıştırın, zombiyi bulun, sonra *ebeveyni* `kill PID` ile sonlandırın. Zombiye ne olur ve neden?

13 Tekrar Soruları

Kavramsal

1. Kabuk `sleep 10` ve `sleep 10 &` için neyi farklı yapar? `fork()`, `exec()` ve `wait()` sözcüklerini kullanın.
2. 3. haftanın beş durumunun her birini `ps`'in gösterdiği harflerle eşleştirin. Hangi Linux durumunun 3. haftada karşılığı yoktur?
3. `kill 3` ile `kill %3` arasındaki fark nedir?
4. SIGTERM ile SIGKILL'i karşılaştırın. Neden önce SIGTERM denenmelidir? Hangi durumlarda SIGKILL bile bir süreci listeden kaldırmaz?
5. `fg` ve `bg` durdurulmuş bir işe ne gönderir ve aralarındaki fark nedir?
6. Sıradan bir kullanıcı nice değerini neden yükseltebilir ama düşüremez?

Sayısal

7. Bir iş bitti ve `echo $?` 130 yazdırıyor. Ne oldu? 129 ve 137 ne anlama gelirdi?
8. 4 çekirdekli bir makine `load average: 6.00, 3.00, 1.00` gösteriyor. Her değeri yorumlayın. Yük artıyor mu, azalıyor mu?
9. İşlemci yoğun iki süreç tek bir çekirdeği paylaşıyor; birinin nice değeri 0 (ağırlık 1024), ötekininki 5 (ağırlık 335). İşlemci paylarını hesaplayın.
10. `top`'taki bir süreç `NI 19` gösteriyor. PR değeri nedir? Nice değeri -20 olan bir sürecin PR değeri nedir?
11. Bir kabuğun dört işi var. `yes > /dev/null &` komutundan sonra sırasıyla `sleep 50 &`, `sleep 60 &`, `sleep 70` (ardından Ctrl+Z) yazıldı. Her birinin iş numarası nedir ve geçerli iş hangisidir?

Sayısal soruların yanıtları

7. $130 = 128 + 2$: SIGINT (Ctrl+C) ile sonlandırıldı. $129 = 128 + 1$: SIGHUP (hat kapandı). $137 = 128 + 9$: SIGKILL.
8. 4 çekirdeğe göre: son dakika $6/4 = 150\%$ (ortalama $6 - 4 = 2$ süreç işlemci bekliyor); son 5 dakika $3/4 = 75\%$; son 15 dakika $1/4 = 25\%$. 1 dakikalık değer en büyüğüdür: yük artıyor.
9. $1024/(1024 + 335) = 1024/1359 \approx 75.3\%$ ve $335/1359 \approx 24.7\%$; oran $1024/335 \approx 3.06 \approx 1.25^5 \approx 3.05$.
10. PR $= 20 + 19 = 39$; nice -20 için: PR $= 20 - 20 = 0$.
11. `yes [1]`, `sleep 50 [2]`, `sleep 60 [3]`, `sleep 70 [4]` (durdurulmuş). Geçerli iş (+) en son durdurulan ya da başlatılan iştir, yani `[4]`; `[3]` - ile işaretlidir.

14 İleriye Bakış: 14. Hafta

Gelecek hafta komutları dosyalara koyup programa dönüştürüyoruz: **Kabuk Betiklerine Giriş** (shell scripting). Shebang satırı olan betikler yazacak, onları `chmod +x` ile çalıştırılabilir yapacak; değişkenleri, tırnaklamayı, komut yerine koymayı `$( )`, aritmetiği `$(( ))`, konumsal parametreleri, `read`'i, çıkış durumu `$?`'i, testleri, `if` deyimini, döngüleri ve fonksiyonları kullanacaksınız. Bu haftanın konuları hemen geri dönüyor: arka plan işlerini yönetmek için `$$` ve `$!`, onları beklemek için `wait` ve bir betik bir komutun neden başarısız olduğunu denetlerken $128 + n$ kuralı. 15. hafta final sınavı tekrarıdır.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 3 (Processes), özellikle Kısım 3.1–3.3 (süreç kavramı, zamanlama, süreçler üzerindeki işlemler) ve Linux zamanlaması (nice değerleri) üzerine Kısım 5.7.
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Bölüm 4, "The Abstraction: The Process": `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf`
  - Bölüm 5, "Interlude: Process API" (fork, exec, wait, sinyaller, `ps`, `top`, `kill`): `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`

- GNU Bash kılavuzu, “Job Control” bölümü: https://www.gnu.org/software/bash/manual/html_node/Job-Control.html

15 Laboratuvar Föyü

Hazırlık: `bash mcmp117_lab_setup.sh`, ardından `cd ~/lab`. 13. haftanın `code/` klasöründeki `zombie.c` dosyasını `~/lab/src` içine kopyalayın. Deneme süreci olarak yalnızca `sleep` ve `yes` kullanın ve `student` yerine kendi kullanıcı adınızı yazın. Her alıştırma için komut satır(lar)ını ve ilgili çıktıyı yazın ve soruyu yanıtlayın. Puanlar köşeli parantez içinde, toplam 100. Çözümler Kısım 16’da; önce kendiniz deneyin!

Bölüm A --- Süreçleri incelemek (26 puan)

A1 [4] Kabuğunuzun PID’sini yazdırın ve aynı numarayı `ps` çıktısında bulun.

A2 [5] `ps -f` çalıştırın. `ps` sürecinin PPID’si nedir ve neden `$$`’a eşittir?

A3 [6] `sleep 1000 &` başlatın. `/proc/$!/status` kullanarak `State` ve `PPid` değerlerini bildirin. Bu hangi 3. hafta durumudur?

A4 [5] İki `sleep` işi daha başlatın ve `pstree -p $$` komutunun yazdırdığı ağacı çizin.

A5 [6] `yes > /dev/null &` başlatın ve bu terminaldeki bütün süreçlerin `pid`, `stat` ve `cmd` değerlerini gösterin. `yes`’in neden R, `sleep`’in neden S olduğunu açıklayın.

Bölüm B --- İş denetimi (24 puan)

B1 [6] `sleep 500`’ü ön planda çalıştırın ve Ctrl+Z ile durdurun. Onu `jobs` ile, STAT değerini `ps` ile gösterin.

B2 [6] B1’deki işi arka planda sürdürün. Yeniden çalıştığını `jobs -l` ve `ps` ile kanıtlayın.

B3 [6] Arka planda üç `sleep` işi başlatın. 2 numaralı işi ön plana getirin ve Ctrl+C ile bitirin. `jobs` şimdi ne gösterir?

B4 [6] `kill 1`, ardından `kill %1` yazın. İki farklı sonucu açıklayın.

Bölüm C --- Sinyaller (26 puan)

C1 [8] Üç `sleep 300 &` işini sırasıyla SIGTERM, SIGKILL ve SIGINT ile bitirin. Her birinden sonra `wait $!` ve `echo $?` çalıştırın. Üç sayıyı açıklayın.

C2 [6] Arka plandaki bir `sleep`’i `kill -STOP` ile duraklatın, `kill -CONT` ile sürdürün; her adımdan sonra STAT değerini gösterin.

C3 [6] `sleep 901`, `sleep 902` ve `sleep 903`’ü arka planda başlatın. `sleep` süreçlerinizi `pgrep` ile sayın, hepsini tek bir komutla sonlandırın ve yeniden sayın.

C4 [6] `sleep 800 &` ve `nohup sleep 900 > /dev/null 2>&1 &` başlatın. İki işe de SIGHUP gönderin. Hangisi sağ kalır ve neden?

Bölüm D --- Öncelik, zombiler ve meydan okumalar (24 puan)

D1 [7] `sleep 600`'ü nice değeri 15 ile başlatın ve `NI` ile `STAT` değerlerine bakın. Nice değerini 19'a yükseltin, sonra yeniden 0'a ayarlamayı deneyin. Sonucu açıklayın.

D2 [8] `zombie.c`'yi `gcc -Wall -O2` ile derleyin, `./zombie 30 &` çalıştırın ve zombiyi `ps` ile gösterin. Ona `kill -9` deneyin. Ne olur ve onu sonunda ne kaldırır?

D3 [4] *Meydan okuma.* Tek bir komut satırıyla, `%CPU` değeri en yüksek üç sürecinizi (başlık satırıyla birlikte) gösterin.

D4 [5] *Meydan okuma (12. hafta).* Tek bir boru hattıyla, süreçlerinizden kaçının hangi durumda olduğunu sayın (STAT'ın ilk harfi).

16 Laboratuvar Föyü Çözümleri

Çözümler yeni oluşturulmuş bir laboratuvarda çalıştırıldı. Öğretici kısımlarda zaten görünen çıktılara başvuru yapılıyor; ötekiler (A3, A5, B4, C3, D1, D3, D4) `code/demo_lab.sh` betiğinden gelir ve tam gerçek çıktıları `code/out_lab.txt` dosyasındadır. Sizin PID'leriniz farklı olacaktır. Başka doğru çözümler de vardır; önemli olan doğru sonuç ve mantıklı bir açıklamadır.

	Komut satır(lar)ı	**Beklenen çıktı / yanıt**
A1	`echo $$` `ps`	1913 gibi bir sayı; `ps` içinde bu PID'ye sahip satır `bash`'tir (Kısım 3).
A2	`ps -f`	`ps -f`'in PPID'si kabuğun PID'sine eşittir: `ps`'e dönüşen süreci kabuk fork etti.
A3	`sleep 1000 &` `grep -E '^(State|PPid)' /proc/$!/status`	`State: S (sleeping)`, `PPid:` kabuğun PID'si (`out_lab.txt` içinde 3207). 3. hafta durumu: *bekliyor* (bir zamanlayıcıyı).
A4	`sleep 2000 & sleep 3000 &` `pstree -p $$`	Çocukları `pstree` ve üç `sleep` süreci olan `bash(PID)`, Kısım 3'teki gibi.
A5	`yes > /dev/null &` `ps -o pid,stat,cmd`	`yes` R'dir: her zaman işlemci ister (hazır ya da çalışıyor). `sleep` S'dir: zamanlayıcısını bekler. `ps`'in kendisi `R+`, `-bash` `Ss`'dir.

B1	`sleep 500`, sonra Ctrl+Z
	`jobs`
	`ps -o pid,stat,cmd`
		`[1]+ Stopped sleep 500`; `STAT T` (Kısım 4).

B2	`bg`
	`jobs -l`
	`ps -o pid,stat,cmd`
		`[1]+ sleep 500 &`, sonra `Running`; yeniden `STAT S`.

B3	`sleep 100 & sleep 200 & sleep 300 &`
	`fg %2`, sonra Ctrl+C
	`jobs`
		`jobs` yalnızca 1 ve 3 numaralı işleri `Running` olarak listeler; 2 numaralı iş bitti (`Terminated` iletisi yok: ön planda bitti).

B4	`kill 1` `kill %1`	`-bash: kill: (1) - Operation not permitted`: `kill 1` root'a ait PID 1'i hedefler. `kill %1` 1 numaralı işi sonlandırır (sonraki istemde `Terminated`).

C1	`sleep 300 &`
	`kill $!` (sonra `-9`, `-INT`)
	`wait $!; echo $?`
		$143, 137, 130 = 128+15, 128+9, 128+2$ (Kısım 5).

C2	`sleep 300 &`
	`kill -STOP $!; ps -o pid,stat,cmd -p $!`
	`kill -CONT $!; ps -o pid,stat,cmd -p $!`
		T, sonra S.

	Komut satır(lar)ı	Beklenen çıktı / yanıt

C3	`pgrep -c -u student sleep`
	`pkill -u student sleep`
	`pgrep -c -u student sleep`
		3, üç `Terminated` iletisi, sonra `0`. (Önceki alıştırmalardan kalan işler hâlâ çalışıyorsa ilk sayı daha büyüktür.)

C4	`sleep 800 &`
	`nohup sleep 900 > /dev/null 2>&1 &`
	`kill -HUP %1 %2; jobs`
		`sleep 800`: Hangup. `nohup` işi `Running` olarak sürer: `nohup`, `sleep`'i exec etmeden önce SIGHUP'ı "yok say" olarak ayarladı (Kısım 5.8).

D1	`nice -n 15 sleep 600 &`
	`ps -o pid,ni,stat,cmd -p $!`
	`renice -n 19 -p $!`
	`renice -n 0 -p $!`
		`NI 15`, `STAT SN`; `old priority 15, new priority 19`; sonra `Permission denied`: kullanıcılar nice değerini yalnızca yükseltebilir.

D2	`gcc -Wall -O2 -o zombie zombie.c`
	`./zombie 30 &`
	`ps -o pid,ppid,stat,cmd`
	`kill -9 <çocuk PID>`
		Çocuk `Z` ve `[zombie] <defunct>` gösterir. `kill -9` hiçbir şeyi değiştirmez: süreç zaten ölü. 30 s sonra ebeveynin `waitpid()` çağrısı onu toplar (Kısım 7); ebeveyni öldürmek de onu kaldırırdı (PID 1'in evlat edinmesi).

D3	`ps -u student -o pid,stat,%cpu,cmd` `--sort =-%cpu | head -4`	Başlık artı üç satır; en üstte yaklaşık %98 ile `yes` (`head -4` = başlık + 3).
D4	`ps -u student -o stat = | cut -c1 | sort | uniq -c`	Bizim çalıştırmamızda `2 R` (`yes`, `ps`) ve `5 S` (`-bash`, `sleep 600`, `cut`, `sort`, `uniq`). `stat =` sütunu başlıksız yazdırır; `cut -c1` durum harfini tutar; `sort | uniq -c` sayar (12. hafta). Boru hattının kendi süreçleri de sayılır.